Chapter 5: Retrieving Data from APIs

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to make HTTP GET requests to REST APIs using the Python requests library and interpret HTTP status codes
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to parse nested JSON responses and extract relevant data fields using dictionary and list indexing
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to construct API requests with querystring parameters to filter, search, and paginate results
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to design and implement pagination loops to retrieve complete datasets from APIs across multiple requests
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to convert API responses into Pandas DataFrames for downstream analysis

5.1Introduction

In earlier chapters, you learned how to collect data by creating it yourself or by extracting it from websites using web scraping techniques. While web scraping can be useful, it is often fragile, unstructured, and dependent on the layout of a website that you do not control. In professional analytics workflows, a more reliable and scalable approach is preferred whenever possible.

One of the most common and powerful ways analysts retrieve data today is through Application programming interfaces (APIs) A structured interface that allows one software system to request and receive data from another system in a standardized way.. APIs are intentionally designed to provide clean, documented, and repeatable access to data. Instead of manually downloading files or scraping web pages, analysts can use APIs to automate data collection and integrate external data directly into their analysis pipelines.

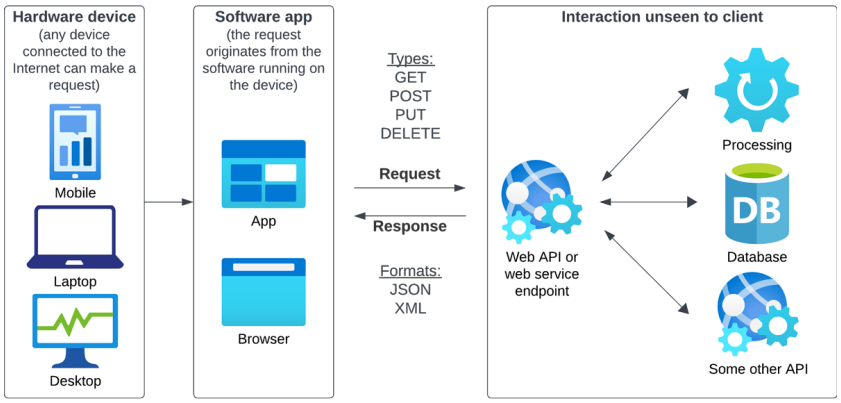
Many modern APIs are delivered as RESTful web services A web-based API style that uses standard HTTP requests and URLs to access structured data resources.. RESTful APIs typically allow users to retrieve data using simple web requests and receive results in structured formats such as JSON. In this chapter, you will primarily work with requests that retrieve data rather than modify it, focusing on how to access and interpret the information returned by an API.

From an analytics perspective, APIs serve as modular data services. Much like building with LEGO bricks, analysts can combine multiple APIs to enrich datasets, update information automatically, or swap out one data source for another with minimal changes to their code. This modular approach supports flexible, distributed data workflows that are common in modern organizations.

A typical API-based workflow involves requesting data from an API endpoint, receiving a response in JSON format, converting that JSON into a Pandas DataFrame, and then applying the same data wrangling, analysis, and modeling techniques you have already learned. Because APIs are designed for programmatic access, they are generally more stable, ethical, and efficient than scraping data from websites that were not intended for automated use.

The examples in this chapter cover public data services, APIs that require authentication keys, and APIs commonly used in business and analytics contexts (health data, financial markets, consumer platforms). The goal is for you to retrieve data from web services and integrate it into your analytics workflows.

5.2Web Service APIs



How Do APIs Work?

Step 1: The Request

An API interaction begins when a client application sends a request to a specific API `endpoint` A URL that represents a specific resource or data service provided by an API.. For analysts, this client is often a Python script or notebook rather than a web or mobile application.

Most analytics-focused API requests use the HTTP GET method to retrieve data. A request always includes the endpoint URL and may also include parameters, headers, or authentication credentials depending on the API.

- **URL**: The address of the API endpoint being requested.
- **Parameters**: Key–value pairs added to the URL to filter, search, or limit the returned data.
- **Headers**: Metadata sent with the request that may specify authentication, data format, or versioning.
- **Payload**: Data sent to the API for processing, typically used with POST or PUT requests rather than data retrieval.

In this chapter, you will primarily work with GET requests that retrieve data rather than modify it.

Step 2: Processing the Request

After receiving the request, the API processes it by validating inputs, checking permissions, and retrieving or computing the requested data. This may involve querying databases, calling internal services, or applying business logic before a response is generated.

From the client's perspective, this processing is invisible. The client simply waits for a response.

Step 3: The Response

Once processing is complete, the API returns a response that includes a status code indicating success or failure, optional headers, and the requested data formatted as JSON or XML. JSON has become the dominant format because it is compact, readable, and easy to work with in Python.

APIs are widely used because they allow organizations to centralize data and logic while supporting many different applications and users. Compared to web scraping, APIs are typically more stable, efficient, and ethically appropriate for automated data access.

API Methods

APIs support several HTTP request methods. While analytics work typically relies on GET requests, it is helpful to understand the broader set of methods.

Table 5.1

Method	Purpose	Analytics Context Example
GET	Retrieve data from an API.	Download daily stock prices or public health statistics.
POST	Submit data for processing.	Send text to a sentiment analysis API.
PUT	Update existing data.	Update stored preferences in an application.
DELETE	Remove data.	Delete an existing record from a system.

Data Formats

APIs typically exchange data using structured, self-describing formats. The two most common formats are JSON and XML, both of which can represent nested, hierarchical data.

JSON is now the most widely used format for APIs because it is lightweight, readable, and maps naturally to Python dictionaries and lists. XML is still used in some legacy systems but is less common in modern analytics workflows.

JSON Example

JSON, like most languages, ignores white space. Therefore, it can be written on a single line or nested on multiple lines as in the two examples below:


```
{&quot;employees&quot;:[{&quot;firstName&quot;:&quot;Suzy&quot;,&quot;lastName&quot;:&quot;Smith&quot;},
{&quot;firstName&quot;:&quot;Bob&quot;,&quot;lastName&quot;:&quot;Jones&quot;},
{&quot;firstName&quot;:&quot;Abigail&quot;,&quot;lastName&quot;:&quot;Adams&quot;}]}
```

```
{
  &quot;employees&quot;: [
    {
      &quot;firstName&quot;: &quot;Suzy&quot;,
      &quot;lastName&quot;: &quot;Smith&quot;
    },
    {
      &quot;firstName&quot;: &quot;Bob&quot;,
      &quot;lastName&quot;: &quot;Jones&quot;
    },
    {
      &quot;firstName&quot;: &quot;Abigail&quot;,
      &quot;lastName&quot;: &quot;Adams&quot;
    }
  ]
}
```


XML Example


```
&lt;employees&gt;&lt;employee&gt;&lt;firstName&gt;Suzy&lt;/firstName&gt;&lt;lastName&gt;Smith&lt;/lastName&gt;&lt;/employee&gt;&lt;
employee&gt;&lt;firstName&gt;Bob&lt;/firstName&gt;&lt;lastName&gt;Jones&lt;/lastName&gt;&lt;/employee&gt;&lt;/employees&gt;
```


```
&lt;employees&gt;
  &lt;employee&gt;
    &lt;firstName&gt;Suzy&lt;/firstName&gt;
    &lt;lastName&gt;Smith&lt;/lastName&gt;

  &lt;/employee&gt;
  &lt;employee&gt;
    &lt;firstName&gt;Bob&lt;/firstName&gt;
    &lt;lastName&gt;Jones&lt;/lastName&gt;

  &lt;/employee&gt;
  &lt;employee&gt;
    &lt;firstName&gt;Abigail&lt;/firstName&gt;
    &lt;lastName&gt;Adams&lt;/lastName&gt;

  &lt;/employee&gt;

&lt;/employees&gt;
```

Authentication Methods

Many APIs require authentication to control access, monitor usage, and prevent abuse. Authentication determines who is allowed to make requests and how frequently.

- **No Authentication**: Open APIs that allow unrestricted access, which are increasingly rare.
- **Basic Authentication**: Uses a username and password; simple but generally discouraged due to security risks.
- **API Keys**: A unique identifier passed with requests that allows providers to track usage and enforce rate limits.
- **OAuth**: A token-based system used when accessing user-specific or sensitive data.

Public APIs

Many APIs are publicly accessible and provide high-quality data suitable for analysis. These APIs often require registration (and sometimes an API key) but do not require payment for limited usage. In this section, you will explore a curated list of options and then review additional APIs that are especially useful for analytics projects.

The Public APIs GitHub repository maintains a curated list of free APIs across many domains. You can browse it here: https://github.com/public-apis/public-apis.

Table 1 includes (1) 10 interesting options drawn from the GitHub Public APIs collection, plus (2) additional APIs for sports (excluding ESPN), games, books, finance, and general analytics datasets. Always review each API's usage notes, rate limits, and authentication requirements before building a large data pull.

Table 5.2
Selected Public APIs for Analytics Projects (30+ Options)

API Name	Domain	Base Endpoint	Documentation	Auth
USGS Earthquake (FDSN Event)	Science & Risk	https://earthquake.usgs.gov/fdsnws/event/1/query	USGS FDSN Event API Docs	No
NASA APIs	Science	https://api.nasa.gov/	NASA API Portal	Key
Open-Meteo	Weather	https://api.open-meteo.com/v1/	Open-Meteo Docs	No
Open Library	Books	https://openlibrary.org/	Open Library API Docs	No
CoinGecko	Finance (Crypto)	https://api.coingecko.com/api/v3/	CoinGecko API Docs	No
Nominatim (OpenStreetMap)	Geocoding	https://nominatim.openstreetmap.org/	Nominatim API Overview	No
OpenAQ	Environment	https://api.openaq.org/	OpenAQ Docs	Varies
Wikipedia REST API	Knowledge	https://en.wikipedia.org/api/rest_v1/	Wikipedia REST API	No

API Name	Domain	Base Endpoint	Documentation	Auth
balldontlie	Sports (Basketball)	https://api.balldontlie.io/v1/	balldontlie Docs	Varies
TheSportsDB	Sports (Multi-league)	https://www.thesportsdb.com/api/	TheSportsDB API Docs	Key
Ergast Developer API	Sports (F1 Motorsport)	http://ergast.com/api/f1/	Ergast MRData Docs	No
OpenF1	Sports (F1 Telemetry)	https://api.openf1.org/v1/	OpenF1 Docs	No
MLB StatsAPI	Sports (Baseball)	https://statsapi.mlb.com/api/	MLB StatsAPI Docs	No
NHL Stats API	Sports (Hockey)	https://statsapi.web.nhl.com/api/	Community NHL API Reference	No
OpenLigaDB	Sports (Soccer)	https://www.openligadb.de/api/	OpenLigaDB Docs	No
RAWG Video Games Database	Games	https://api.rawg.io/api/	RAWG API Docs	Key
CheapShark	Games (Deals)	https://www.cheapshark.com/api/	CheapShark API Docs	No
FreeToGame	Games	https://www.freetogame.com/api/	FreeToGame API Docs	No

API Name	Domain	Base Endpoint	Documentation	Auth
BoardGameGeek API (XML)	Games	https://boardgamegeek.com/xmlapi2/	BGG XML API2 Docs	No
TMDB (The Movie Database)	Entertainment	https://api.themoviedb.org/3/	TMDB Docs	Key
Gutendex (Project Gutenberg)	Books	https://gutendex.com/	Gutendex Docs / Examples	No
Google Books	Books	https://www.googleapis.com/books/v1/	Google Books API Docs	Key
Crossref	Research	https://api.crossref.org/	Crossref REST API	No
OpenAlex	Research	https://api.openalex.org/	OpenAlex Docs	No
Wikidata	Knowledge Graph	https://query.wikidata.org/	Wikidata SPARQL Service	No
Alpha Vantage	Finance (Markets)	https://www.alphavantage.co/query	Alpha Vantage Docs	Key
FRED (St. Louis Fed)	Finance (Macro)	https://api.stlouisfed.org/fred/	FRED API Docs	Key
SEC EDGAR	Finance (Filings)	https://data.sec.gov/	SEC API Docs	No

API Name	Domain	Base Endpoint	Documentation	Auth
Frankfurter	Finance (FX)	https://api.frankfurter.app/	Frankfurter Docs	No
ExchangeRate.host	Finance (FX)	https://api.exchangerate.host/	ExchangeRate.host Docs	No
World Bank API	Economics	https://api.worldbank.org/v2/	World Bank Indicators API Docs	No
JSONPlaceholder	Teaching / Mock Data	https://jsonplaceholder.typicode.com/	JSONPlaceholder	No
HTTP Cat	Teaching	https://http.cat/	HTTP Cat	No
IPify	Utilities	https://api.ipify.org	IPify Docs	No
Zippopotam.us	Geography	https://api.zippopotam.us/	Zippopotam.us	No
Open-Movie / OMDb	Entertainment	https://www.omdbapi.com/	OMDb API Docs	Key
Open Trivia Database	Entertainment	https://opentdb.com/api.php	OpenTDB API Docs	No
Stack Exchange	Developer / Q&A	https://api.stackexchange.com/2.3/	Stack Exchange API Docs	Key (

API Name	Domain	Base Endpoint	Documentation	Auth
GitHub REST API	Developer / Open Source	https://api.github.com/	GitHub REST API Docs	Toker (reco
OpenStreetMap Overpass	Geospatial	https://overpass-api.de/api/	Overpass API	No
Data.gov Catalog API	Government / Open Data	https://catalog.data.gov/api/	Data.gov CKAN API Docs	No

5.3In Python

Basic APIs

Python has several libraries for calling web APIs to retrieve data. This chapter uses the *requests* package, which you previously used for web scraping.

The simplest APIs are free and require no authentication, headers, parameters, or request body data. These are ideal for learning because they let you focus on the mechanics of making requests and interpreting responses without additional setup.

Because open APIs can change or go offline without notice, it is always good practice to check the status code and inspect the response before assuming the data is valid. Later in this chapter, you will learn how to handle failures more robustly.

- https://randomuser.me/api/ — Returns demographic and profile information for a randomly generated user.
- https://dog.ceo/api/breeds/image/random — Returns a random dog image URL.
- https://official-joke-api.appspot.com/random_joke — Returns a random programming joke.
- https://api.zippopotam.us/us/33162 — Returns geographic information for a U.S. ZIP code.
- https://ipinfo.io/161.185.160.93/geo — Returns geographic metadata for an IP address.
- https://isro.vercel.app/api/spacecrafts — Returns a list of spacecraft launched by ISRO.
- https://catfact.ninja/fact — Returns a random cat fact (simple JSON key “fact”).
- https://www.boredapi.com/api/activity — Suggests a random activity (good for practice with non-nested JSON).
- https://api.ipify.org?format=json — Returns the client’s public IP address as JSON.

- https://www.themealdb.com/api/json/v1/1/random.php — Returns a random meal recipe with structured JSON.

The example below demonstrates how to make a simple GET request using the Random User API.

```
import requests

# '.get' refers to the request method being used (GET, POST, PUT, DELETE, etc.)
response = requests.get(&quot;https://randomuser.me/api/&quot;)
print(response.status_code)


# Output:
# 200
```

A status code of **200** indicates that the request was successful and the server returned data. The table below summarizes some common status codes, what they mean, and likely causes.

Table 5.3
Common HTTP Status Codes, Meanings, and Likely Causes

Status Code	Meaning	Explanation	Likely Causes
200	OK	The request was successful and the server returned the expected data.	Valid endpoint, correct request format, and proper authentication (if required).
301	Moved Permanently	The requested resource has been permanently moved to a new URL.	Outdated endpoint URL or an API version change.
400	Bad Request	The server could not understand the request due to invalid syntax or parameters.	Missing or malformed query parameters, invalid data types, or an incorrect request body.
401	Unauthorized	The request requires authentication, and valid credentials were not provided.	Missing API key, expired token, or incorrect credentials.
403	Forbidden	The server understood the request but refuses to authorize it.	Insufficient permissions, restricted resource, or an API key that lacks the required scope.
404	Not Found	The server could not find the requested resource.	Incorrect endpoint path, deleted resource, or a typo in the URL.
429	Too Many Requests	The client has sent too many requests in a given amount of time.	Rate limits exceeded on free or restricted APIs.
500	Internal Server Error	The server encountered an unexpected condition that prevented it from fulfilling the request.	Server-side bugs, crashes, or temporary outages.
503	Service Unavailable	The server is currently unable to handle the request.	API downtime, maintenance windows, or overloaded servers.

To see the raw data returned by the API, access the *.text* attribute of the response object.

```
response.text


# Output:
# {&quot;results&quot;:[{&quot;gender&quot;:&quot;female&quot;,&quot;name&quot;:
{&quot;title&quot;:&quot;Miss&quot;,&quot;first&quot;:&quot;Gretl&quot;,&quot;last&quot;:&quot;Wiegmann&quot;},&quot;location&quot;
:{&quot;street&quot;:{&quot;number&quot;:6445,&quot;name&quot;:&quot;Neue
Straße&quot;},&quot;city&quot;:&quot;Gedern&quot;,&quot;state&quot;:&quot;Thüringen&quot;,&quot;country&quot;:&quot;Germany&quot;,&
quot;postcode&quot;:82979,&quot;coordinates&quot;:
```

{"latitude":"-49.5994","longitude":"111.6012"},"timezone":
{"offset":"-9:00","description":"Alaska"}},"email":"gretl.wiegmann@example.c
om","login":{"uuid":"5f11ab4f-c868-42d0-8636-
3cf1fb61dfb5","username":"happyelephant814","password":"chicago1","salt":&qu
ot;FyGj1dsy","md5":"e446e6c2997ef9684a409d6f31166011","sha1":"069d9e06c02c3e65f8eb5669d4416
6c9e7e9f0b7","sha256":"ab7c3f62396d4499a983e1ca16931b0d1d2379e0e71a8dd051c49f661f877b9c"},"dob":
{"date":"1998-07-06T22:52:25.360Z","age":27},"registered":{"date":"2003-09-
08T07:57:57.166Z","age":22},"phone":"0740-8973298","cell":"0170-
8616661","id":{"name":"SVNR","value":"73 060798 W 744"},"picture":
{"large":"https://randomuser.me/api/portraits/women/96.jpg","medium":"https://randomuser.me/api/
portraits/med/women/96.jpg","thumbnail":"https://randomuser.me/api/portraits/thumb/women/96.jpg"},"na
t":"DE"}],"info":
{"seed":"fea5b9de92752393","results":1,"page":1,"version":"1.4"}}

The returned data is formatted as JSON, which is a structured, text-based representation of nested dictionaries and lists. Python provides tools to convert JSON strings into native data structures.

```
import json

json_data = json.loads(response.text)

print(json.dumps(json_data, indent=2))
```

Okay, that helps. JSON is basically a complex Python dictionary of nested lists and dictionaries in the form of key/value pairs. For example, the first label "results" is a key, while the list that appears on the other side of the ":" symbol is the value for that key even though that list contains one and only one item: a very large dictionary containing information about the generated person.

Our goal is to extract the relevant fields from this JSON object and convert them into a small table that we can store and export as a .CSV file. The first step is understanding how to drill down through a JSON object.

```
# Start with the entire json_data object
# Then go into the value of the 'results' key
# Then go to the first item in the list (index 0), which is a dictionary
# Then go to the 'location' key of that dictionary and return the value (another dictionary)
json_data['results'][0]['location']


# Output:
# {'street': {'number': 6445, 'name': 'Neue Straße'},
#  'city': 'Gedern',
#  'state': 'Thüringen',
#  'country': 'Germany',
#  'postcode': 82979,
#  'coordinates': {'latitude': '-49.5994', 'longitude': '111.6012'},
#  'timezone': {'offset': '-9:00', 'description': 'Alaska'}}
```

Notice how each part (['results'], [0], and ['location']) drills one level deeper into the JSON object. This is important to understand, so ask your instructor for help if you are still not sure. Lists are accessed with positions (e.g., 0), while dictionaries are accessed by key names (e.g., 'results' and 'location'). Both use the square bracket notation []. This is different from how lists and dictionaries are created ([] for lists, {} for dictionaries).

With this drill-down technique, you can now build a Pandas DataFrame by creating a loop that calls the API multiple times and adds one row per user.

```
import pandas as pd
import json
import requests

df = pd.DataFrame(columns=['name', 'gender', 'age', 'city', 'state'])
df.set_index('name', inplace=True)
```

```
# Generate several users
for i in range(5):
  response = requests.get(&quot;https://randomuser.me/api/&quot;)
  json_data = json.loads(response.text)
  name = json_data['results'][0]['name']['first'] + &quot; &quot; + json_data['results'][0]['name']['last']

  df.loc[name] = [json_data['results'][0]['gender'],
                  json_data['results'][0]['dob']['age'],
                  json_data['results'][0]['location']['city'],
                  json_data['results'][0]['location']['state']]

# Save the results to a .CSV file and print the DataFrame
df.to_csv('users.csv')
df.head()


# Output:
#                   gender      age         city            state
# name
# Hudson Park   male    54      Radisson        British Columbia
# Gonzalo Bravo male    68      Móstoles        Andalucía
# Nicolas Ortega        male    32      Santander       Ceuta
# Matias Huotari        male    41      Varkaus Southern Savonia
# Afet Özbey    female  49      Yozgat  Karabük
```

Remember that you will have different data in this table than the example because the API returns randomly generated users. Now you have a dataset that you can work with. It only has a few rows, but the code is dynamic and will work whether there are 3 or 3000+ rows.

Endpoints

You may have noticed a term used above: *endpoint*. The type of APIs we are using here is a specific type referred to as *REST web services*. Web services have a URL (e.g., https://www.domainname.com) and one or more endpoints (e.g., https://www.domainname.com/api/**endpointname**). An endpoint The location from which each of the functions offered by REST Web Service APIs can be accessed. is a specific functionality offered by a RESTful web service. Each endpoint has its own name, which is simply attached to the web service URL, thus giving each endpoint its own location.

Every web service API will have its own documentation detailing each available endpoint, the method for calling it (e.g., GET or POST), inputs required, and outputs delivered. Consider another web service from the earlier list: https://isro.vercel.app/api. This API has four endpoints that provide data about spacecraft, launchers, customer satellites, and ISRO centres. The code below demonstrates each one.

```
import requests

# Returns a list of ISRO launched spacecraft
response = requests.get(&quot;https://isro.vercel.app/api/spacecrafts&quot;)
print(response.json())

# Returns a list of ISRO launchers
response = requests.get(&quot;https://isro.vercel.app/api/launchers&quot;)
print(response.json())

# Returns a list of ISRO customer satellites
response = requests.get(&quot;https://isro.vercel.app/api/customer_satellites&quot;)
print(response.json())

# Returns a list of ISRO centres
response = requests.get(&quot;https://isro.vercel.app/api/centres&quot;)
print(response.json())


# Output:
# {'spacecrafts': [{'id': 1, 'name': 'Aryabhata'}, {'id': 2, 'name': 'Bhaskara-I'}, {'id': 3, 'name': 'Rohini Technology
Payload (RTP)'} ...
# {'launchers': [{'id': 'SLV-3E1'}, {'id': 'SLV-3E2'}, {'id': 'SLV-3D1'}, {'id': 'SLV-3'}, {'id': 'ASLV-D1'}, {'id': 'ASLV-
D2'}, {'id':  ...
# {'customer_satellites': [{'id': 'DLR-TUBSAT', 'country': 'Germany', 'launch_date': '26-05-1999', 'mass': '45',
```

```
'launcher': 'PSLV-C2'}, ...
        # {'centres': [{'id': 1, 'name': 'Semi-Conductor Laboratory (SCL)', 'Place': 'Chandigarh', 'State': 'Punjab/Haryana'},
{'id': 2, 'name': ...
```

Before moving on, notice that we did not call json.loads() to convert the string in the response object into JSON. This is because the requests package provides a response method named **.json()** that parses JSON for you. You will see both techniques used in practice, so you should be familiar with both.

Parameters (Querystring)

Web services can be customized in a few ways. Querystring parameters are one of those ways. Parameters are placed in the URL by adding a '?' symbol at the end and then one or more key/value pairs separated by '&' between each pair (e.g., key=value&key=value&key=value). The examples below demonstrate free web services that require parameters.

```
        import requests

        # Predict the age of a person based on their name
        response = requests.get(&quot;https://api.agify.io?name=homer&quot;)
        print(response.text)

        # Predict the gender of a person based on their name
        response = requests.get(&quot;https://api.genderize.io?name=homer&quot;)
        print(response.text)

        # Predict the ethnic background of a person based on their name
        response = requests.get(&quot;https://api.nationalize.io?name=homer&quot;)
        print(response.text)

        # Get the current weather for a lat/lon location
        response = requests.get(&quot;https://api.open-meteo.com/v1/forecast?
latitude=40.23&amp;longitude=-111.66&amp;current_weather=true&quot;)
        print(response.text)

        # Get your IP address
        response = requests.get(&quot;https://api.ipify.org?format=json&quot;)
        print(response.text)

        # Get game deals from Cheapshark
        response = requests.get(&quot;https://www.cheapshark.com/api/1.0/deals?storeID=1&amp;upperPrice=15&quot;)
        print(response.text)


        # Output:
        # {&quot;count&quot;:2238,&quot;name&quot;:&quot;homer&quot;,&quot;age&quot;:73}
        #
{&quot;count&quot;:9109,&quot;name&quot;:&quot;homer&quot;,&quot;gender&quot;:&quot;male&quot;,&quot;probability&quot;:0.99}
        # {&quot;count&quot;:6669,&quot;name&quot;:&quot;homer&quot;,&quot;country&quot;:
[{&quot;country_id&quot;:&quot;US&quot;,&quot;probability&quot;:0.1501604815598518},
{&quot;country_id&quot;:&quot;GB&quot;,&quot;probability&quot;:0.12146996887761972},
{&quot;country_id&quot;:&quot;TT&quot;,&quot;probability&quot;:0.07829013637114275},
{&quot;country_id&quot;:&quot;NZ&quot;,&quot;probability&quot;:0.04305957500412852},
{&quot;country_id&quot;:&quot;CA&quot;,&quot;probability&quot;:0.037852248941131886}]}
        #
{&quot;latitude&quot;:40.230103,&quot;longitude&quot;:-111.65572,&quot;generationtime_ms&quot;:0.09083747863769531,&quot;utc_offset
_seconds&quot;:0,&quot;timezone&quot;:&quot;GMT&quot;,&quot;timezone_abbreviation&quot;:&quot;GMT&quot;,&quot;elevation&quot;:1384.
0,&quot;current_weather_units&quot;:
{&quot;time&quot;:&quot;iso8601&quot;,&quot;interval&quot;:&quot;seconds&quot;,&quot;temperature&quot;:&quot;
°C&quot;,&quot;windspeed&quot;:&quot;km/h&quot;,&quot;winddirection&quot;:&quot;
°&quot;,&quot;is_day&quot;:&quot;&quot;,&quot;weathercode&quot;:&quot;wmo code&quot;},&quot;current_weather&quot;:
{&quot;time&quot;:&quot;2026-01-
01T23:30&quot;,&quot;interval&quot;:900,&quot;temperature&quot;:7.5,&quot;windspeed&quot;:19.5,&quot;winddirection&quot;:115,&quot;
is_day&quot;:1,&quot;weathercode&quot;:3}}
        # {&quot;ip&quot;:&quot;35.229.123.83&quot;}
        # [{&quot;internalName&quot;:&quot;DISCOELYSIUMTHEFINALCUT&quot;,&quot;title&quot;:&quot;Disco Elysium - The Final
Cut&quot;,&quot;metacriticLink&quot;:&quot;\/game\/disco-elysium-the-final-
cut\/&quot;,&quot;dealID&quot;:&quot;Uk9pH81%2BgPUwbkh9YzwMnYLT%2B%2FEaktqN8AgwMr6Y2wQ%3D&quot;,&quot;storeID&quot;:&quot;1&quot;,&
quot;gameID&quot;:&quot;227942&quot;,&quot;salePrice&quot;:&quot;3.99&quot;,&quot;normalPrice&quot;:&quot;39.99&quot;,&quot;isOnSal
e&quot;:&quot;1&quot;,&quot;savings&quot;:&quot;90.022506&quot;,&quot;metacriticScore&quot;:&quot;97&quot;,&quot;steamRatingText&qu
ot;:&quot;Very
Positive&quot;,&quot;steamRatingPercent&quot;:&quot;92&quot;,&quot;steamRatingCount&quot;:&quot;53822&quot;,&quot;steamAppID&quot;:
&quot;632470&quot;,&quot;releaseDate&quot;:1617062400,&quot;lastChange&quot;:1766082481,&quot;dealRating&quot;:&quot;9.2&quot;,&quo
t;thumb&quot;:&quot;https:\/\/shared.fastly.steamstatic.com\/store_item_assets\/steam\/apps\/632470\/capsule_231x87.jpg?
t=1766855203&quot;},...]
```

5.4Earthquakes Example

In this case study, you will retrieve earthquake event data from the United States Geological Survey (USGS) Earthquake API. Many organizations (for example, emergency management agencies, insurers, logistics providers, and utility companies) monitor seismic activity to support risk assessment and operational planning. Your goal is to make multiple requests to a single endpoint, paginate through results using querystring parameters, assemble the returned records into a Pandas DataFrame, save the dataset to a CSV file, and create a visualization that summarizes the full dataset.

The USGS endpoint supports querystring parameters that let you filter by time window and control pagination. Two parameters are especially important here: **limit** (how many records to return per request) and **offset** (how many records to skip before returning the next page). By increasing the offset in a loop, you can retrieve the complete dataset for your chosen time window in multiple API calls.

The following endpoint returns JSON in GeoJSON format:

https://earthquake.usgs.gov/fdsnws/event/1/query

Before you begin, choose a time range that returns a manageable number of records for classroom use (for example, a few days or a couple of weeks). If you choose a very large time range, you may retrieve thousands of events, which will take longer to download and process.

1. Make one request to verify the endpoint works and inspect the JSON structure.
2. Identify where the list of earthquake events is stored in the JSON response.
3. Write a pagination loop that repeatedly calls the endpoint using **limit** and **offset** until no more results are returned.
4. Extract a consistent set of fields from each event and append them into a Python list of dictionaries.
5. Convert the list of dictionaries into a Pandas DataFrame and save the dataset as a CSV file.
6. Create a Seaborn visualization based on the full dataset (not just one page of results).

Step 1: Make a single request and inspect the response. The code below requests events in a time window and prints the status code. Then it prints the top-level keys so you can see the structure of the returned JSON object.

```
import requests
import json

base_url = &quot;https://earthquake.usgs.gov/fdsnws/event/1/query&quot;

params = {
  &quot;format&quot;: &quot;geojson&quot;,
  &quot;starttime&quot;: &quot;2024-01-01&quot;,
  &quot;endtime&quot;: &quot;2024-01-08&quot;,
  &quot;minmagnitude&quot;: 2.5,
  &quot;limit&quot;: 200,
  &quot;offset&quot;: 1
}

response = requests.get(base_url, params=params)
print(response.status_code)
json_data = response.json()
print(list(json_data.keys()))


# Output:
```

```
# 200
# ['type', 'metadata', 'features', 'bbox']
```

Step 2: Find where the earthquake records live. In the GeoJSON response, the events are stored in a list under the **features** key. Print the length of that list and preview one record so you can see what fields are available.

```
features = json_data[&quot;features&quot;]
print(len(features))
print(json.dumps(features[0], indent=2)[:1200])


# Output:
# 200
# {
#   &quot;type&quot;: &quot;Feature&quot;,
#   &quot;properties&quot;: {
#     &quot;mag&quot;: 2.76,
#     &quot;place&quot;: &quot;0 km NNW of Prattville, CA&quot;,
#     &quot;time&quot;: 1704667606430,
#     &quot;updated&quot;: 1710020404040,
#     &quot;tz&quot;: null,
#     &quot;url&quot;: &quot;https://earthquake.usgs.gov/earthquakes/eventpage/nc73986226&quot;,
#     &quot;detail&quot;: &quot;https://earthquake.usgs.gov/fdsnws/event/1/query?eventid=nc73986226&amp;format=geojson&quot;,
#     &quot;felt&quot;: 1,
#     &quot;cdi&quot;: 2,
#     &quot;mmi&quot;: null,
#     &quot;alert&quot;: null,
#     &quot;status&quot;: &quot;reviewed&quot;,
#     &quot;tsunami&quot;: 0,
#     &quot;sig&quot;: 117,
#     &quot;net&quot;: &quot;nc&quot;,
#     &quot;code&quot;: &quot;73986226&quot;,
#     &quot;ids&quot;: &quot;,nc73986226,us6000m2dq,&quot;,
#     &quot;sources&quot;: &quot;,nc,us,&quot;,
#     &quot;types&quot;: &quot;,dyfi,nearby-cities,origin,phase-data,scitech-link,&quot;,
#     &quot;nst&quot;: 44,
#     &quot;dmin&quot;: 0.03814,
#     &quot;rms&quot;: 0.18,
#     &quot;gap&quot;: 38,
#     &quot;magType&quot;: &quot;md&quot;,
#     &quot;type&quot;: &quot;earthquake&quot;,
#     &quot;title&quot;: &quot;M 2.8 - 0 km NNW of Prattville, CA&quot;
#   },
#   &quot;geometry&quot;: {
#     &quot;type&quot;: &quot;Point&quot;,
#     &quot;coordinates&quot;: [
#       -121.1591667,

#       40.211,

#       2.61
#     ]
#   },
#   &quot;id&quot;: &quot;nc73986226&quot;
# }
```

Step 3: Build a pagination loop. The idea is to request a fixed number of records per call (**limit**) and then move through the dataset by increasing **offset**. When a request returns an empty **features** list, you have reached the end of the available results for the chosen filters.

Some APIs enforce rate limits. Even when an API is free and open, you should avoid sending requests as fast as possible. In the loop below, a short sleep is included as a polite practice and to reduce the chance of temporary errors.

```
import time

all_rows = []
limit = 200
offset = 1

while True:
  params = {
```

```
    &quot;format&quot;: &quot;geojson&quot;,
    &quot;starttime&quot;: &quot;2024-01-01&quot;,
    &quot;endtime&quot;: &quot;2024-01-08&quot;,
    &quot;minmagnitude&quot;: 2.5,
    &quot;limit&quot;: limit,
    &quot;offset&quot;: offset
  }

  response = requests.get(base_url, params=params)

  if response.status_code != 200:
    print(&quot;Request failed with status code:&quot;, response.status_code)
    break

  page = response.json()
  features = page[&quot;features&quot;]

  if len(features) == 0:
    break

  for f in features:
    props = f.get(&quot;properties&quot;, {})
    geom = f.get(&quot;geometry&quot;, {})
    coords = geom.get(&quot;coordinates&quot;, [None, None, None])

    row = {
      &quot;event_id&quot;: f.get(&quot;id&quot;),
      &quot;time_ms&quot;: props.get(&quot;time&quot;),
      &quot;place&quot;: props.get(&quot;place&quot;),
      &quot;magnitude&quot;: props.get(&quot;mag&quot;),
      &quot;event_type&quot;: props.get(&quot;type&quot;),
      &quot;url&quot;: props.get(&quot;url&quot;),
      &quot;tsunami_flag&quot;: props.get(&quot;tsunami&quot;),
      &quot;longitude&quot;: coords[0],
      &quot;latitude&quot;: coords[1],
      &quot;depth_km&quot;: coords[2]
    }

    all_rows.append(row)

  offset = offset + limit
  time.sleep(0.25)

print(&quot;Total events collected:&quot;, len(all_rows))


# Output:
# Total events collected: 513
```

Step 4: Convert the results into a DataFrame and perform basic cleanup. The API provides time in milliseconds since the Unix epoch. Converting that value into a datetime makes analysis and plotting much easier.

```
import pandas as pd

df = pd.DataFrame(all_rows)
df[&quot;time_utc&quot;] = pd.to_datetime(df[&quot;time_ms&quot;], unit=&quot;ms&quot;, utc=True)
df.drop(columns=[&quot;time_ms&quot;], inplace=True)
df[&quot;magnitude&quot;] = pd.to_numeric(df[&quot;magnitude&quot;], errors=&quot;coerce&quot;)
df[&quot;depth_km&quot;] = pd.to_numeric(df[&quot;depth_km&quot;], errors=&quot;coerce&quot;)
df.head()
```

	event_id	place	magnitude	event_type	url	tsunami_flag	longitude	latitude	depth_km	time_utc
0	nc73986226	0 km NNW of Prattville, CA	2.76	earthquake	https://earthquake.usgs.gov/earthquakes/eventp...	0	-121.159167	40.2110	2.6100	2024-01-07 22:46:46.430000+00:00
1	us6000m422	50 km NNW of Atka, Alaska	4.20	earthquake	https://earthquake.usgs.gov/earthquakes/eventp...	0	-174.441200	52.6279	176.8790	2024-01-07 22:18:17.078000+00:00
2	tx2024amqs	45 km NW of Toyah, Texas	2.50	earthquake	https://earthquake.usgs.gov/earthquakes/eventp...	0	-104.109000	31.6180	7.2242	2024-01-07 22:04:21.549000+00:00
3	us6000m420	129 km WSW of Gorontalo, Indonesia	4.10	earthquake	https://earthquake.usgs.gov/earthquakes/eventp...	0	121.920500	0.2941	186.7010	2024-01-07 21:52:24.514000+00:00
4	ak024bxk9sb	89 km W of Akhiok, Alaska	2.90	earthquake	https://earthquake.usgs.gov/earthquakes/eventp...	0	-155.624600	56.8555	6.1000	2024-01-07 21:50:09.986000+00:00

Step 5: Save the dataset. You now have a complete dataset built from multiple API calls. Save it to a CSV file so it can be reused for later analysis.

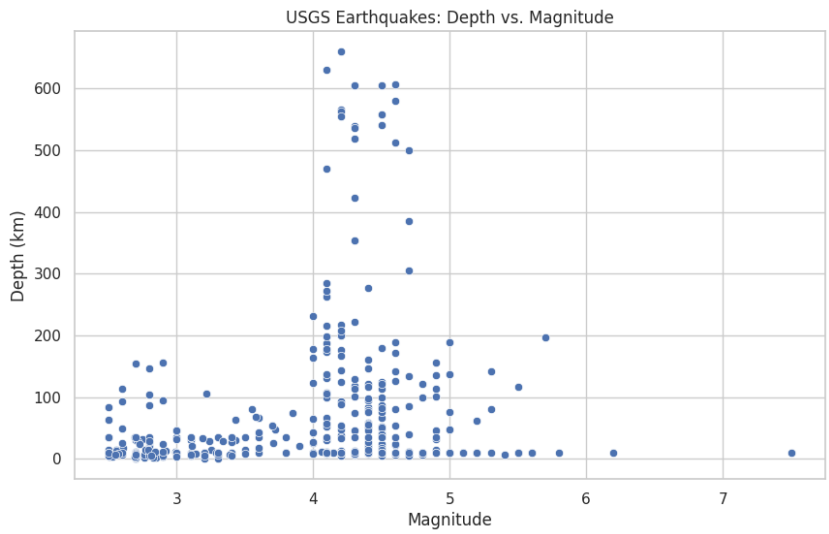
```
df.to_csv(&quot;usgs_earthquakes.csv&quot;, index=False)
print(df.shape)


# Output (varies by date range):
# (9564, 5)
```

Step 6: Create a visualization in Seaborn using the full dataset. The example below creates a scatter plot showing how earthquake depth relates to magnitude. Because the plot is based on the DataFrame you assembled from all pages, it summarizes the entire dataset, not just the first API response.

```
import seaborn as sns
from matplotlib import pyplot as plt

plot_df = df.dropna(subset=[&quot;magnitude&quot;, &quot;depth_km&quot;])
plt.figure(figsize=(10, 6))
sns.scatterplot(data=plot_df, x=&quot;magnitude&quot;, y=&quot;depth_km&quot;)
plt.title(&quot;USGS Earthquakes: Depth vs. Magnitude&quot;)
plt.xlabel(&quot;Magnitude&quot;)
plt.ylabel(&quot;Depth (km)&quot;)
plt.show()
```



At this point, you have demonstrated an end-to-end workflow for retrieving data from a web service API, handling pagination with querystring parameters, assembling a dataset, saving it, and visualizing the results. This same approach generalizes to many real-world APIs that return large datasets across multiple pages of results.

5.5Key-Based Authentication

Free APIs are convenient for learning, but many professional-quality datasets and services require authentication. One of the most common approaches is a key-based system, where the API provider issues you a unique key and expects you to include it with each request. This allows the provider to track usage, enforce rate limits, and (when applicable) bill for service.

In this section, you will learn how to use a simple querystring API key with a popular weather service. You will create an account, confirm your email address, generate an API key, and then include that key in the URL when making a request.

Below you'll find instructions on how to sign up for an API key. This is not required. You are welcome to simply review the code below to see how the process works without running the code. Or, you can follow the instructions to generate a key for yourself to experiment with the OpenWeather API.

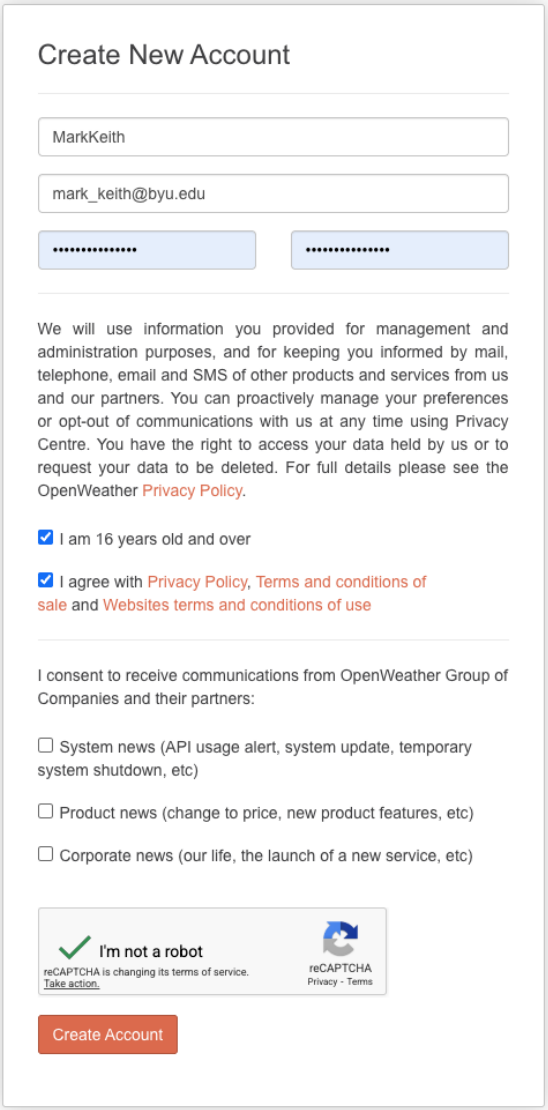
Important: After you create your account and generate your API key, it may take up to **2 hours** for the key to become active. During this activation window, you may receive a **401** status code with a message such as **"Invalid API key"** even if you copied the key correctly and confirmed your email. This is a normal delay on the provider side, not a Python error. If you receive a 401 error shortly after signing up, wait 15–30 minutes and try again, and continue retrying periodically for up to 2 hours. It took me ~20 minutes before my new key worked.

Querystring/Parameter Keys

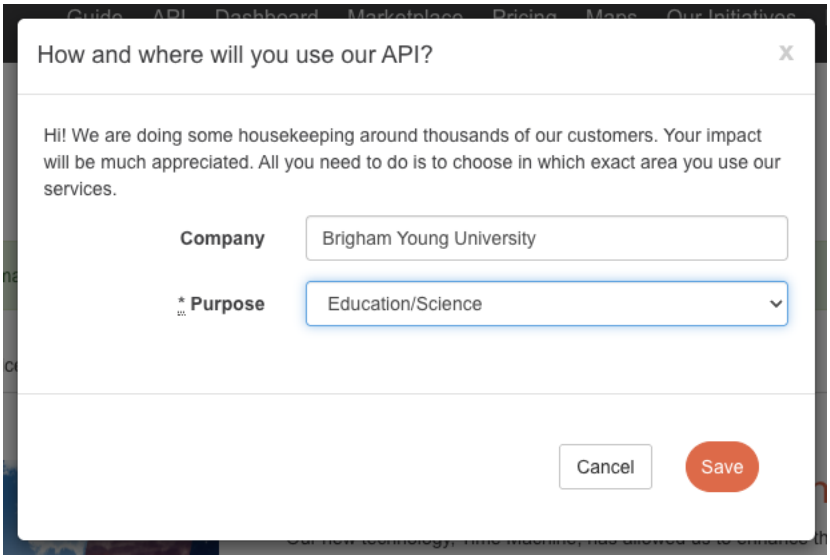
The simplest form of authentication involves placing a key or token in the URL querystring as a `request parameter` Data sent to a URL in the querystring in a dictionary (?key=value) format.. In this example, the weather provider uses an API key parameter named **appid**. Your request will look like a normal URL, but it will include your API key at the end.

Follow the steps below to create your account and retrieve your API key. Use the screenshots provided in the chapter to ensure your screens match what is shown.

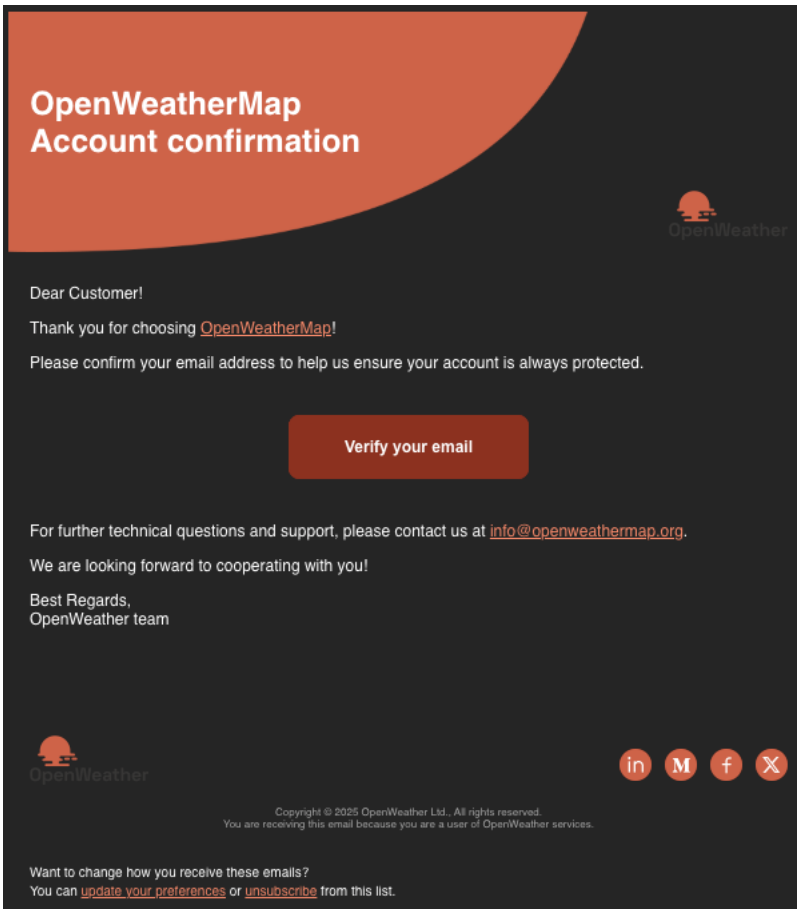
1. **Step 1:** Create your account at the provider’s sign-up page and submit the required information.



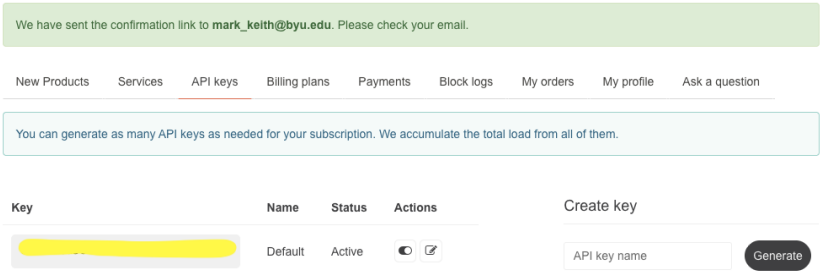
2. **Step 2:** During registration (or immediately after), you may be asked a question similar to “How and where will you use our API?” Select the option that best matches your intended learning use (for example, educational or personal learning). Then submit the form.



3. **Step 3:** Confirm your email address. After you sign up, the provider will send a confirmation email. Open the message and click the confirmation link. Your API key may not function until email confirmation is complete.



4. **Step 4:** Log in to your dashboard and navigate to the API keys view. Copy your key exactly as shown (API keys are case-sensitive). Store it securely and do not share it publicly.



Once you have your key, you will include it in the request URL as the value of the **appid** parameter. The example below requests the current weather for Provo, Utah. Note that **q** (the city and country code) and **units** are also querystring parameters.

```
import requests
import json

key = '****************************************'  # Paste your key over these stars; keep the quotes

# Build the request URL using querystring parameters:
# q = city,country_code
# units = imperial or metric
# appid = your API key

url = (
  &quot;https://api.openweathermap.org/data/2.5/weather&quot;
  &quot;?q=Provo,US&quot;  # Change this to any city/state combination you'd like to try
  &quot;&amp;units=imperial&quot;
  &quot;&amp;appid=&quot; + key
)

response = requests.get(url)
print(response.status_code)
json_data = json.loads(response.text)
clean_data = json.dumps(json_data, indent=2)

print(clean_data)


# Output:
# 200
# {
#   &quot;coord&quot;: {
#     &quot;lon&quot;: -111.6585,
#     &quot;lat&quot;: 40.2338
#   },
#   &quot;weather&quot;: [
#     {
#       &quot;id&quot;: 804,
#       &quot;main&quot;: &quot;Clouds&quot;,
#       &quot;description&quot;: &quot;overcast clouds&quot;,
#       &quot;icon&quot;: &quot;04n&quot;
#     }
#   ],
#   &quot;base&quot;: &quot;stations&quot;,
#   &quot;main&quot;: {
#     &quot;temp&quot;: 44.04,
#     &quot;feels_like&quot;: 42.66,
#     &quot;temp_min&quot;: 41.25,
#     &quot;temp_max&quot;: 46.06,
#     &quot;pressure&quot;: 1015,
#     &quot;humidity&quot;: 90,
#     &quot;sea_level&quot;: 1015,
#     &quot;grnd_level&quot;: 835
#   },
#   &quot;visibility&quot;: 10000,
#   &quot;wind&quot;: {
#     &quot;speed&quot;: 3.27,
#     &quot;deg&quot;: 112,
#     &quot;gust&quot;: 4
#   },
#   &quot;clouds&quot;: {
```

```
#     &quot;all&quot;: 100
#   },
#   &quot;dt&quot;: 1767314539,
#   &quot;sys&quot;: {
#     &quot;type&quot;: 2,
#     &quot;id&quot;: 2009866,
#     &quot;country&quot;: &quot;US&quot;,
#     &quot;sunrise&quot;: 1767278950,
#     &quot;sunset&quot;: 1767312662
#   },
#   &quot;timezone&quot;: -25200,
#   &quot;id&quot;: 5780026,
#   &quot;name&quot;: &quot;Provo&quot;,
#   &quot;cod&quot;: 200
# }
```

Placing a key in a querystring has an important security implication: the key is included in the URL. If a URL is logged, shared, or captured in plain text, the key could be exposed. In modern practice, most reputable APIs require HTTPS (secure sockets layer encryption), which encrypts the request in transit. Even so, you should treat API keys as sensitive credentials: do not post them in screenshots, GitHub repositories, or discussion forums.

If you receive a **401** status code and a message indicating an invalid API key, double-check that you copied the key correctly (no extra spaces) and that your email is confirmed. If the key is new, remember that activation may take up to **2 hours**. If the problem persists beyond that window, regenerate a new key in the dashboard and try again.

The JSON response includes many fields, but you will usually only need a subset. For example, the **main** sub-dictionary contains numeric measures such as temperature and humidity, while the **weather** list contains a human-readable description of conditions.

```
# Extract a few useful fields from the returned JSON
city = json_data.get('name')
temp_f = json_data.get('main', {}).get('temp')
humidity = json_data.get('main', {}).get('humidity')
condition = json_data.get('weather', [{}])[0].get('description')

print(city, temp_f, humidity, condition)


# Output: Provo 44.04 90 overcast clouds
```

The remaining sections of this chapter cover additional authentication patterns (including headers and OAuth), more robust error handling, and techniques for retrieving larger datasets by making multiple calls and assembling the results into a DataFrame.

5.6Header and Body Data

In the previous section, you learned how some APIs use a querystring key (added directly to the URL) for authentication. While that approach is common, many professional web services use a more secure pattern: the client sends authentication information in the request **headers** and sends input data in the request **body**. This is especially important for machine learning prediction services, where you must send structured feature values to the model and receive a prediction in return.

A widely used approach for header-based authentication is OAuth An open standard for delegated access that allows clients to authenticate securely without placing secrets in the URL.. In many modern APIs, you do not include a key in the querystring. Instead, you include a token in the request headers (often as a bearer token) so

the credential is not exposed in the URL. When you need to send larger or more complex inputs than simple query parameters, you place those inputs in the request body.

The example below previews the kind of API call you will need to make for your final course project: a **POST** request to a prediction endpoint. The request includes (1) a JSON-formatted request body containing feature values for one or more customers, and (2) an **Authorization** header containing a bearer token. The web service returns predictions in a JSON response, which we then parse and print.

Note: This example uses a pre-deployed demo web service so you can focus on the structure of the request. In your final project, you will deploy your own prediction service and generate your own key or token.

```
import requests
import json

# Define the input data structure
data = {
  &quot;Inputs&quot;: {
    &quot;input1&quot;: [  # This is a list of feature sets; you can send multiple customers at a time
      { # Example of a customer's data
        &quot;age&quot;: 19,              # Age of the customer
        &quot;sex&quot;: &quot;female&quot;,        # Gender of the customer
        &quot;bmi&quot;: 27.9,            # BMI (Body Mass Index) of the customer
        &quot;children&quot;: 0,          # Number of children the customer has
        &quot;smoker&quot;: &quot;yes&quot;,        # Smoking status: &quot;yes&quot; or &quot;no&quot;
        &quot;region&quot;: &quot;southwest&quot;   # Geographic region: northeast, northwest, southeast, southwest
      },
      {  # Collect user inputs for another customer
        &quot;age&quot;: int(input(&quot;What is your age? (enter a valid integer): &quot;)),
        &quot;sex&quot;: input(&quot;What is your gender? (female/male): &quot;),
        &quot;bmi&quot;: float(input(&quot;What is your BMI? (e.g., 21.3): &quot;)),
        &quot;children&quot;: int(input(&quot;How many children do you have? (enter an integer): &quot;)),
        &quot;smoker&quot;: input(&quot;Do you smoke? (enter yes or no): &quot;),
        &quot;region&quot;: input(&quot;Where do you live? (options: northeast, northwest, southeast, southwest): &quot;)
      } # You could potentially add as many customers as you want to with additional dictionaries here
    ]
  }
}

# Encode the data as a JSON string
body = str.encode(json.dumps(data))

# Define the web service endpoint and API key
url = 'https://YOUR-ENDPOINT-ID.westcentralus.azurecontainer.io/score'  # Replace with your endpoint URL
api_key = 'YOUR_API_KEY_HERE'  # Replace with your actual API key

# Set the headers for the HTTP request
headers = {
  'Content-Type': 'application/json',  # Specify the format of the request body
  'Authorization': 'Bearer ' + api_key  # Include the API key for authorization
}

# Send the POST request to the endpoint; the result is immediately parsed as JSON for further use
req = requests.post(url=url, data=body, headers=headers).json()

# Print the raw response for debugging purposes
print('\nRaw response from the API:', req, '\n')

# Iterate through the response results to extract and display projected costs
print(&quot;Customer projected costs:&quot;)

for result in req['Results']['WebServiceOutput0']:
  # Format and right-align the printed cost value
  print(f&quot;  Customer projected cost: ${result['Scored Labels']:&gt;10.2f}&quot;)


# Output:
#  What is your age? (enter a valid integer): 45
#  What is your gender? (female/male): male
#  What is your BMI? (e.g., 21.3): 24.3
#  How many children do you have? (enter an integer): 3
#  Do you smoke? (enter yes or no): no
#  Where do you live? (options: northeast, northwest, southeast, southwest): southeast
#
#  Raw response from the API: {'Results': {'WebServiceOutput0': [{'Scored Labels': 25180.178081707592}, {'Scored Labels':
7992.073217857426}]}}
#
#  Customer projected costs:
#    Customer projected cost: $  25180.18
```

```
#    Customer projected cost: $   7992.07
```

Notice three important features of this request. First, the method is **POST**, which is commonly used when you are sending input data to be processed (such as feature values for a prediction). Second, the request body is a JSON document containing a list of records. This allows you to score multiple customers in a single request. Third, the authentication credential is not placed in the URL. Instead, it is included in the **Authorization** header as a bearer token, which is the pattern you will commonly see in production APIs.

In real projects, you should avoid hard-coding API keys directly in your notebook or script. A simple improvement is to store your key in a separate local file and read it at runtime. This reduces the chance of accidentally sharing the key when you submit code, upload notebooks, or copy/paste snippets.

```
# Example: store your API key in a separate file and load it at runtime
#
# Create a file named: api_key.txt
# Put ONLY the key on a single line, like this:
# YOUR_API_KEY_HERE
with open(&quot;api_key.txt&quot;, &quot;r&quot;) as f:
  api_key = f.read().strip()

headers = {
  &quot;Content-Type&quot;: &quot;application/json&quot;,
  &quot;Authorization&quot;: &quot;Bearer &quot; + api_key
}

# You can now reuse 'headers' in your POST request:
# req = requests.post(url=url, data=body, headers=headers).json()
```

Later, you will learn stronger approaches (such as environment variables or secret managers). However, using a separate key file is an easy first step that keeps sensitive values out of your code while still being simple to understand and use.

5.7Stock Market Data Example

Once you are comfortable working with APIs, a wide range of real-world data sources becomes accessible. Financial markets are a good example. In this section, you will retrieve historical stock price data using the polygon.io API. Polygon provides professional-grade market data through a well-documented REST API and is widely used in analytics, trading, and fintech applications.

To follow along, you will need to create a free Polygon account and generate an API key. The free tier is sufficient for this example and does not require any payment information. You will use the API key as a bearer token in the request headers, reinforcing the header-based authentication pattern introduced earlier in this chapter.

The video below demonstrates how to make authenticated requests to Polygon's API and interpret the returned JSON data. After watching the video, review the provided code examples to see how the same logic can be implemented directly in Python.

This video can be viewed online.

The first example below demonstrates how to retrieve daily aggregate price data for a single stock ticker (Apple, AAPL) over a short date range. Notice that the API key is passed in the request headers rather than in the querystring.

```
import json, requests

key = &quot;YOUR_POLYGON_API_KEY&quot;  # Replace with your own key from polygon.io
headers = {&quot;Authorization&quot;: &quot;Bearer &quot; + key}
url = &quot;https://api.polygon.io/v2/aggs/ticker/AAPL/range/1/day/2023-08-17/2023-08-18?
adjusted=true&amp;sort=asc&amp;limit=120&quot;
req = requests.get(url, headers=headers)
result = json.loads(req.text)
result


# Output:
# {
#   'ticker': 'AAPL',
#   'queryCount': 1,
#   'resultsCount': 1,
#   'adjusted': True,
#   'results': [
#     {
#       'v': 66054482.0,
#       'vw': 174.5857,
#       'o': 177.14,
#       'c': 174.0,
#       'h': 177.5054,
#       'l': 173.48,
#       't': 1692244800000,
#       'n': 673083
#     }
#   ],
#   'status': 'DELAYED',
#   'request_id': '...',
#   'count': 1
# }
```

The response contains both metadata and a **results** list with price and volume fields such as open, high, low, close, and volume. These values are timestamped and suitable for time-series analysis.

The next example expands on this idea by making multiple API calls (one per trading day) and appending the results into a Pandas DataFrame. This pattern is common when building datasets from APIs that return one day or one page of data per request.

```
import json, requests, pandas as pd

dates = ['2023-08-01', '2023-08-02', '2023-08-03', '2023-08-04', '2023-08-05']
key = &quot;YOUR_POLYGON_API_KEY&quot;  # Replace with your own key from polygon.io
headers = {&quot;Authorization&quot;: &quot;Bearer &quot; + key}

df = pd.DataFrame(columns=[
  'date', 'symbol', 'open', 'high', 'low', 'close',
  'volume', 'afterHours', 'preMarket'
])

for date in dates:
  url = f&quot;https://api.polygon.io/v1/open-close/AAPL/{date}?adjusted=true&quot;
  req = requests.get(url, headers=headers)
  result = json.loads(req.text)
  try:
    df.loc[len(df)] = [
      result['from'], result['symbol'], result['open'], result['high'],
      result['low'], result['close'], result['volume'],
      result['afterHours'], result['preMarket']
    ]

  except (KeyError, ValueError):
    print(result)

df
```

```
# Output:
# {'status': 'NOT_FOUND', 'request_id': '69dcb26ce3eac136ad58422b9d660428', 'message': 'Data not found.'}
# Note that you'll only get that message above if you attempt more than 5 API calls in a minute
```

If you exceed the free-tier rate limit, the API may return an error message instead of data. This highlights the importance of handling exceptions when working with production APIs. After building the DataFrame, you can export the data to a CSV file, visualize trends, or integrate the dataset into a larger analytics workflow.

	date	symbol	open	high	low	close	volume	afterHours	preMarket
0	2023-08-01	AAPL	196.235	196.73	195.2800	195.605	35281426.0	195.30	196.35
1	2023-08-02	AAPL	195.040	195.18	191.8507	192.580	50388811.0	192.81	193.56
2	2023-08-03	AAPL	191.570	192.37	190.6900	191.170	62243282.0	187.30	191.90
3	2023-08-04	AAPL	185.520	187.38	181.9200	181.990	115956841.0	182.05	187.31

5.8ESPN NBA Example

In many real-world analytics workflows, you need completed events with final scores and reliable statistics, not upcoming ones. In this case study, you will use a free ESPN endpoint (no registration and no API key required) to retrieve NBA game data, filter to games that have already been played, and then enrich your dataset with team-level statistics from a second endpoint.

You will build two datasets: (1) a **games** table with one row per completed game, and (2) a **team_stats** table with two rows per game (one row per team). You will then save the results to .CSV files and create a visualization in Seaborn.

These ESPN endpoints are publicly accessible, but they are not officially documented and may change over time. Always check the **status_code** and inspect the JSON structure before assuming it will always look the same.

The primary endpoint for the NBA scoreboard is shown below. By default, it can include a mixture of completed games, in-progress games, and scheduled games (including games that have not happened yet).

Scoreboard endpoint: https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard

Scoreboard with date filter: https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard?dates=YYYYMMDD

Game summary endpoint (per game):
https://site.web.api.espn.com/apis/site/v2/sports/basketball/nba/summary?event=EVENT_ID

Step 1: Inspect the Default Scoreboard Response

Before applying any filters, first call the scoreboard endpoint without parameters. This allows you to confirm that scheduled games are returned by default and explains why additional filtering is necessary.

```
import requests
import json

url = &quot;https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard&quot;
```

```
response = requests.get(url)

print(response.status_code)
json_data = json.loads(response.text)

# Show the first event so we can inspect its status.
events = json_data.get(&quot;events&quot;, [])
events[0]


# Output:
# 200
# {'id': '401810331',
#  'uid': 's:40~l:46~e:401810331',
#  'date': '2026-01-03T00:00Z',
#  'name': 'San Antonio Spurs at Indiana Pacers',
#  ...
#  'status': {'type': {'name': 'STATUS_SCHEDULED',
#                      'state': 'pre',
#                      'completed': False,
#                      'description': 'Scheduled'}}}
```

Step 2: Retrieve the Last 200 Completed Games

The scoreboard endpoint supports a **dates** querystring parameter. We treat each date as a “page” of results, walking backward day-by-day and collecting only completed games until we reach 200.

Completed games typically have a status **state** value of **post**. Filtering on this value ensures that scores and statistics are final.

```
import datetime as dt
import time
import pandas as pd
import requests

BASE = &quot;https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard&quot;

def fetch_scoreboard(date_yyyymmdd):
  url = BASE + &quot;?dates=&quot; + date_yyyymmdd
  r = requests.get(url)

  if r.status_code != 200:
    return None, r.status_code

  return r.json(), 200

completed_games = []
target_n = 200
d = dt.datetime.now(dt.UTC).date()

while len(completed_games) &lt; target_n:
  date_str = d.strftime(&quot;%Y%m%d&quot;)
  data, status = fetch_scoreboard(date_str)

  if status == 200 and data:
    for e in data.get(&quot;events&quot;, []):
      if e.get(&quot;status&quot;, {}).get(&quot;type&quot;, {}).get(&quot;state&quot;) == &quot;post&quot;:
        completed_games.append(e)

  d = d - dt.timedelta(days=1)
  time.sleep(0.25)

completed_games = completed_games[:target_n]
print(&quot;Completed games collected:&quot;, len(completed_games))
print(&quot;Example event id:&quot;, completed_games[0].get(&quot;id&quot;))


# Output:
# Completed games collected: 200
# Example event id: 401810326
```

Step 3: Create the Games DataFrame

Now convert the list of completed games into a structured DataFrame with one row per game.

```
games_df = pd.DataFrame(columns=[
  &quot;event_id&quot;,
  &quot;date_utc&quot;,
  &quot;game_name&quot;,
  &quot;status&quot;,
  &quot;home_team&quot;,
  &quot;away_team&quot;,
  &quot;home_score&quot;,
  &quot;away_score&quot;,
  &quot;venue&quot;
])

for e in completed_games:
  event_id = e.get(&quot;id&quot;)
  date_utc = e.get(&quot;date&quot;)
  game_name = e.get(&quot;name&quot;)
  status_obj = e.get(&quot;status&quot;, {}).get(&quot;type&quot;, {})
  status = status_obj.get(&quot;description&quot;)
  competitions = e.get(&quot;competitions&quot;, [])

  if len(competitions) == 0:
    continue

  comp = competitions[0]
  venue = comp.get(&quot;venue&quot;, {}).get(&quot;fullName&quot;)
  competitors = comp.get(&quot;competitors&quot;, [])
  home = next((c for c in competitors if c.get(&quot;homeAway&quot;) == &quot;home&quot;), None)
  away = next((c for c in competitors if c.get(&quot;homeAway&quot;) == &quot;away&quot;), None)

  if home is None or away is None:
    continue

  home_team = home.get(&quot;team&quot;, {}).get(&quot;displayName&quot;)
  away_team = away.get(&quot;team&quot;, {}).get(&quot;displayName&quot;)
  try:
    home_score = int(home.get(&quot;score&quot;, 0))
  except (TypeError, ValueError):
    home_score = None

  try:
    away_score = int(away.get(&quot;score&quot;, 0))
  except (TypeError, ValueError):
    away_score = None

  games_df.loc[len(games_df)] = [
    event_id,
    date_utc,
    game_name,
    status,
    home_team,
    away_team,
    home_score,
    away_score,
    venue
  ]

games_df.head()
```

	event_id	date_utc	game_name	status	home_team	away_team	home_score	away_score	venue
0	401810326	2026-01-01T23:00Z	Houston Rockets at Brooklyn Nets	Final	Brooklyn Nets	Houston Rockets	96	120	Barclays Center
1	401810327	2026-01-02T00:00Z	Miami Heat at Detroit Pistons	Final	Detroit Pistons	Miami Heat	112	118	Little Caesars Arena
2	401810328	2026-01-02T01:30Z	Philadelphia 76ers at Dallas Mavericks	Final	Dallas Mavericks	Philadelphia 76ers	108	123	American Airlines Center
3	401810329	2026-01-02T03:00Z	Boston Celtics at Sacramento Kings	Final	Sacramento Kings	Boston Celtics	106	120	Golden 1 Center
4	401810330	2026-01-02T03:30Z	Utah Jazz at LA Clippers	Final	LA Clippers	Utah Jazz	118	101	Intuit Dome

Step 4: Enrich the Dataset with Team Statistics

Enrich your dataset by calling the **summary** endpoint for each completed game. This produces a second DataFrame with one row per team per game (two rows per game). Because this step makes many requests (up to 200), include a delay so you do not overwhelm the service.

Before you write a loop that requests 200 games and extracts team statistics, you should first inspect the JSON structure returned by the ESPN *summary* endpoint for a single event. This is an important habit when working with APIs because the data is often nested and the keys may not match what you expect. By printing a small piece of the response (instead of the entire JSON), you can confirm where the team information and the team statistics are located and what the statistics objects look like.

In the code below, we select the first **event_id** from *games_df*, call the summary endpoint, and then extract the first team object in the boxscore. We print the team name and the first five entries in the team's statistics list. This will show you which keys the API uses for statistics (for example, you may see fields such as **abbreviation**, **name**, and **displayValue**). Once you understand this structure, you can confidently write code to extract the statistics you need for every team in every game.

```
event_id = games_df.loc[0, &quot;event_id&quot;]
summary_url = &quot;https://site.web.api.espn.com/apis/site/v2/sports/basketball/nba/summary?event=&quot; + str(event_id)
summary = requests.get(summary_url).json()
team0 = summary[&quot;boxscore&quot;][&quot;teams&quot;][0]
team0[&quot;team&quot;][&quot;displayName&quot;], team0[&quot;statistics&quot;][:5]


# Output:
# ('Houston Rockets',
#  [{'name': 'fieldGoalsMade-fieldGoalsAttempted',
#    'displayValue': '47-82',
#    'label': 'FG'},
#   {'name': 'fieldGoalPct',
#    'displayValue': '57',
#    'abbreviation': 'FG%',
#    'label': 'Field Goal %'},
#   {'name': 'threePointFieldGoalsMade-threePointFieldGoalsAttempted',
#    'displayValue': '12-29',
#    'label': '3PT'},
#   {'name': 'threePointFieldGoalPct',
#    'displayValue': '41',
#    'abbreviation': '3P%',
#    'label': 'Three Point %'},
#   {'name': 'freeThrowsMade-freeThrowsAttempted',
#    'displayValue': '14-18',
#    'label': 'FT'}])
```

Notice that it includes some summary statistics, but not all that we might expect. For example, the score or total points scored by the team is not included in the team statistics list. This happens because ESPN often stores points in a different part of the JSON (typically in the header competitors list), while the boxscore statistics list focuses on shooting, rebounds, assists, turnovers, and related metrics. This is common in real APIs: values you logically group together may be stored in different branches of the JSON structure.

```
import time
import requests
import pandas as pd

team_stats_df = pd.DataFrame(columns=[
  &quot;event_id&quot;,
  &quot;date_utc&quot;,
  &quot;team&quot;,
  &quot;home_away&quot;,
  &quot;points&quot;,
  &quot;fg&quot;,
  &quot;fg_pct&quot;,
  &quot;three_pt&quot;,
  &quot;three_pt_pct&quot;,
  &quot;ft&quot;,
  &quot;ft_pct&quot;,
  &quot;rebounds&quot;,
  &quot;assists&quot;,
  &quot;turnovers&quot;
])
```

```
for idx, row in games_df.iterrows():
  event_id = row[&quot;event_id&quot;]
  date_utc = row[&quot;date_utc&quot;]

  summary_url = (
    &quot;https://site.web.api.espn.com/apis/site/v2/sports/basketball/nba/summary&quot;
    &quot;?event=&quot; + str(event_id)
  )

  res = requests.get(summary_url)

  if res.status_code != 200:
    print(&quot;Skipping event&quot;, event_id, &quot;status&quot;, res.status_code)
    time.sleep(0.35)
    continue

  summary = res.json()

  # 1) Build a points lookup from the HEADER (most reliable location for points).
  #    This is usually present for completed games even when boxscore stats vary.
  points_by_team_id = {}
  try:
    competitors = summary[&quot;header&quot;][&quot;competitions&quot;][0][&quot;competitors&quot;]

    for c in competitors:
      team_id = c.get(&quot;team&quot;, {}).get(&quot;id&quot;)
      score = c.get(&quot;score&quot;)  # usually a string like &quot;112&quot;

      if team_id is not None:
        points_by_team_id[str(team_id)] = score
  except Exception:
    competitors = []

  # 2) Pull team rows from the BOXSCORE and match to header points using team id.
  teams = summary.get(&quot;boxscore&quot;, {}).get(&quot;teams&quot;, [])

  if not teams:
    print(&quot;No boxscore teams found for event&quot;, event_id)
    time.sleep(0.35)
    continue

  for t in teams:
    team_obj = t.get(&quot;team&quot;, {})
    team_name = team_obj.get(&quot;displayName&quot;)
    home_away = t.get(&quot;homeAway&quot;)
    team_id = team_obj.get(&quot;id&quot;)
    points = points_by_team_id.get(str(team_id)) if team_id is not None else None
    # Parse the rest from statistics (keep it explicit)
    stats_list = t.get(&quot;statistics&quot;, [])
    stats = {}

    for s in stats_list:
      key = s.get(&quot;abbreviation&quot;) or s.get(&quot;label&quot;) or s.get(&quot;name&quot;)

      if key is None:
        continue

      val = s.get(&quot;displayValue&quot;)

      if val is None:
        val = s.get(&quot;value&quot;)

      stats[str(key).upper()] = val

    fg = stats.get(&quot;FG&quot;)                                # &quot;47-82&quot;
    fg_pct = stats.get(&quot;FG%&quot;)                           # &quot;57&quot;
    three_pt = stats.get(&quot;3PT&quot;) or stats.get(&quot;3P&quot;)        # &quot;12-29&quot;
    three_pt_pct = stats.get(&quot;3P%&quot;) or stats.get(&quot;3PT%&quot;)  # &quot;41&quot;
    ft = stats.get(&quot;FT&quot;)                                # &quot;14-18&quot;
    ft_pct = stats.get(&quot;FT%&quot;)                           # sometimes missing
    rebounds = stats.get(&quot;REB&quot;) or stats.get(&quot;TRB&quot;)
    assists = stats.get(&quot;AST&quot;)
    turnovers = stats.get(&quot;TO&quot;) or stats.get(&quot;TOV&quot;)

    team_stats_df.loc[len(team_stats_df)] = [
      event_id,
      date_utc,
      team_name,
      home_away,
      points,
      fg,
      fg_pct,
      three_pt,
      three_pt_pct,
      ft,
      ft_pct,
      rebounds,
      assists,
      turnovers
    ]
```

```
    time.sleep(0.35)

team_stats_df.head()
```

	event_id	date_utc	team	home_away	points	fg	fg_pct	three_pt	three_pt_pct	ft	ft_pct	rebounds	assists	turnovers
0	401810326	2026-01-01T23:00Z	Houston Rockets	away	120	47-82	57	12-29	41	14-18	78	45	28	14
1	401810326	2026-01-01T23:00Z	Brooklyn Nets	home	96	34-76	45	10-30	33	18-30	60	27	18	13
2	401810327	2026-01-02T00:00Z	Miami Heat	away	118	43-99	43	14-39	36	18-20	90	49	25	18
3	401810327	2026-01-02T00:00Z	Detroit Pistons	home	112	38-80	48	9-26	35	27-31	87	40	25	20
4	401810328	2026-01-02T01:30Z	Philadelphia 76ers	away	123	48-90	53	13-36	36	14-16	88	43	22	11

Step 5: Save Your Datasets to CSV Files

Save both datasets to .CSV files so you can reuse them for analysis without calling the API again.

```
games_df.to_csv(&quot;espn_nba_games_last200.csv&quot;, index=False)
team_stats_df.to_csv(&quot;espn_nba_team_stats_last200.csv&quot;, index=False)

print(&quot;Saved files:&quot;)
print(&quot; - espn_nba_games_last200.csv&quot;)
print(&quot; - espn_nba_team_stats_last200.csv&quot;)


# Output:
# Saved files:
#  - espn_nba_games_last200.csv
#  - espn_nba_team_stats_last200.csv
```

Step 6: Visualize the Results in Seaborn

Create a visualization in Seaborn. The example below converts the **points** column to numeric and plots the average points by team. Because the full dataset is large, we summarize by team and then chart only the top 20 teams by average points over the collected games.

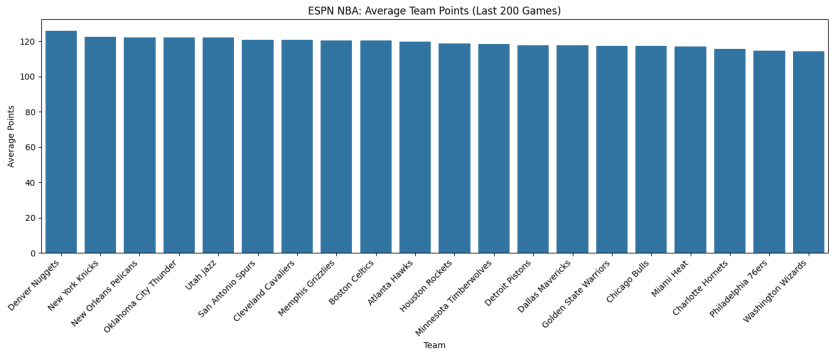
```
import seaborn as sns
from matplotlib import pyplot as plt

team_stats_df[&quot;points_num&quot;] = pd.to_numeric(team_stats_df[&quot;points&quot;], errors=&quot;coerce&quot;)

plot_df = (
  team_stats_df
    .dropna(subset=[&quot;points_num&quot;])
    .groupby(&quot;team&quot;, as_index=False)[&quot;points_num&quot;]
    .mean()
)

# Optional: keep chart readable
plot_df = plot_df.sort_values(&quot;points_num&quot;, ascending=False).head(20)
plt.figure(figsize=(14, 6))
ax = sns.barplot(data=plot_df, x=&quot;team&quot;, y=&quot;points_num&quot;)
plt.xticks(rotation=45, ha=&quot;right&quot;)
plt.title(f&quot;ESPN NBA: Average Team Points (Last {target_n} Games)&quot;)
plt.xlabel(&quot;Team&quot;)
plt.ylabel(&quot;Average Points&quot;)
plt.tight_layout()
plt.show()


# Output:
# (Your chart will appear here.)
```



This example combined several techniques from the chapter: querystring filtering, date-based pagination, enrichment through secondary API calls, DataFrame assembly, and visualization. The same pattern applies to most multi-endpoint API workflows.

Free ESPN Sports Data APIs

If you like this area of sports analytics, ESPN provides a collection of undocumented but publicly accessible web service endpoints that return structured JSON data without requiring API keys, authentication, or student registration. These APIs are commonly used for educational projects, sports analytics prototypes, dashboards, and exploratory data analysis. While not officially guaranteed, they are stable enough for instructional use and allow students to work with real, up-to-date sports data.

These endpoints follow REST-style conventions and return nested JSON structures. They are read-only, rate-limited implicitly, and intended for data retrieval rather than transactional use. Because they are undocumented, students should always inspect the JSON structure before writing full data pipelines.

Table 5.4
Common Free ESPN API Endpoints

API Endpoint	Description
https://site.web.api.espn.com/apis/site/v2/sports	Root endpoint listing all sports supported by ESPN, including basketball, football, baseball, soccer, hockey, and more.
https://site.web.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard	Returns recent, live, and upcoming NBA games including event IDs, teams, scores, game status, and timestamps.
https://site.web.api.espn.com/apis/site/v2/sports/basketball/nba/summary?event={event_id}	Provides detailed box score data for a specific NBA game, including team statistics, player statistics, scoring, and game metadata.
https://site.web.api.espn.com/apis/site/v2/sports/football/nfl/scoreboard	Returns NFL game schedules, scores, and statuses for recent, current, and upcoming games.

API Endpoint	Description
https://site.web.api.espn.com/apis/site/v2/sports/baseball/mlb/scoreboard	Returns Major League Baseball game data including scores, teams, innings, and game states.
https://site.web.api.espn.com/apis/site/v2/sports/hockey/nhl/scoreboard	Returns NHL game schedules and scoring information.
https://site.web.api.espn.com/apis/site/v2/sports/soccer/eng.1/scoreboard	Returns English Premier League soccer match data, including fixtures, scores, and match status.
https://site.web.api.espn.com/apis/site/v2/sports/{sport}/{league}/teams	Returns metadata about teams in a given league, including names, abbreviations, logos, and identifiers.
https://site.web.api.espn.com/apis/site/v2/sports/{sport}/{league}/standings	Returns league standings, rankings, wins/losses, and divisional information.

5.9Practice Data Retrieval

Optionally, it may help to practice the concepts you learned in the last few chapters:

This assessment can be taken online.

5.10Assignment

Complete the assignment below:

This assessment can be taken online.

This assessment can be taken online.